ia_study1_computeMeasures

2026-03-11

Admin (Packages & Custom Functions)

```r
#setwd(dirname(rstudioapi::getActiveDocumentContext()$path))
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.6.0
## v ggplot2   4.0.0     v tibble    3.3.0
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.2.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Load Data

```
data <- read_csv("ia_study1_anon_data.csv")
data <- data[ , -1]
```

Ten Item Personality Inventory (TIPI)

Next, we compute the composite scores of the participant-level TIPI variables. Specifically, we calculate:

1. participant's extraversion `tipi_ex`

- `tipi_1` (Extraverted, enthusiastic), `tipi_6` (reversed; Reserved, quiet)

2. participant's agreeableness, `tipi_a`

- `tipi_2` (reversed; Critical, quarrelsome), `tipi_7` (Sympathetic, warm)

3. participant's conscientiousness, `tipi_c`

- `tipi_3` (Dependable, self-disciplined), `tipi_8` (reversed; Disorganized, careless)

4. participant's emotoinal stability, `tipi_es`

- `tipi_4` (reversed; Anxious, easily upset), `tipi_9` (Calm, emotionally stable)

5. participant's openness to experience, `tipi_o`

- `tipi_5` (Open to new experiences, complex), `tipi_10` (reversed; Conventional, uncreative)

```
# compute tipi variables
data <- data %>%
  rowwise() %>%
  dplyr::mutate( # reverse code tipi items
    tipi_6r = 8 - tipi_6,
    tipi_2r = 8 - tipi_2,
    tipi_8r = 8 - tipi_8,
    tipi_4r = 8 - tipi_4,
    tipi_10r = 8 - tipi_10
    ) %>%
  dplyr::mutate( # calculate tipi_ex, tipi_a, tipi_c, tipi_es, and tipi_o
    tipi_ex = mean(c(tipi_1, tipi_6r), na.rm = T),
    tipi_a = mean(c(tipi_2r, tipi_7), na.rm = T),
    tipi_c = mean(c(tipi_3, tipi_8r), na.rm = T),
    tipi_es = mean(c(tipi_4r, tipi_9), na.rm = T),
    tipi_o = mean(c(tipi_5, tipi_10r), na.rm = T)
    ) %>%
  ungroup()
```

```r
data %>%
  dplyr::select("tipi_1", "tipi_6r",
                "tipi_2r", "tipi_7",
                "tipi_3", "tipi_8r",
                "tipi_4r", "tipi_9",
                "tipi_5", "tipi_10r") %>%
  cor(use = "pairwise.complete.obs") %>%
  round(2)
```

```
##          tipi_1 tipi_6r tipi_2r tipi_7 tipi_3 tipi_8r tipi_4r tipi_9 tipi_5
## tipi_1     1.00    0.62   -0.23   0.10   0.04   -0.22   -0.01  -0.10   0.23
## tipi_6r    0.62    1.00   -0.09   0.02  -0.09   -0.15    0.04  -0.23   0.09
## tipi_2r   -0.23   -0.09    1.00   0.12  -0.04    0.26    0.14   0.08   0.02
## tipi_7     0.10    0.02    0.12   1.00   0.03    0.04   -0.15   0.00   0.15
## tipi_3     0.04   -0.09   -0.04   0.03   1.00    0.45    0.10   0.30   0.21
## tipi_8r   -0.22   -0.15    0.26   0.04   0.45    1.00    0.23   0.18  -0.07
## tipi_4r   -0.01    0.04    0.14  -0.15   0.10    0.23    1.00   0.43   0.00
## tipi_9    -0.10   -0.23    0.08   0.00   0.30    0.18    0.43   1.00   0.08
## tipi_5     0.23    0.09    0.02   0.15   0.21   -0.07    0.00   0.08   1.00
## tipi_10r   0.06    0.21    0.08   0.02   0.01    0.07    0.24  -0.14   0.27
##          tipi_10r
## tipi_1       0.06
## tipi_6r      0.21
## tipi_2r      0.08
## tipi_7       0.02
## tipi_3       0.01
## tipi_8r      0.07
## tipi_4r      0.24
## tipi_9      -0.14
## tipi_5       0.27
## tipi_10r     1.00
```

```r
# European American
data %>% dplyr::filter(cultureEA == 1) %>%
  dplyr::select("tipi_1", "tipi_6r",
                "tipi_2r", "tipi_7",
                "tipi_3", "tipi_8r",
                "tipi_4r", "tipi_9",
               "tipi_5", "tipi_10r") %>%
  cor(use = "pairwise.complete.obs") %>%
  round(2)
```

```
##          tipi_1 tipi_6r tipi_2r tipi_7 tipi_3 tipi_8r tipi_4r tipi_9 tipi_5
## tipi_1     1.00    0.58   -0.19   0.20   0.07   -0.16    0.09   0.14   0.24
## tipi_6r    0.58    1.00    0.02  -0.03  -0.05    0.08    0.22  -0.07   0.11
## tipi_2r   -0.19    0.02    1.00   0.14  -0.16    0.18    0.16  -0.18   0.03
## tipi_7     0.20   -0.03    0.14   1.00   0.09    0.01   -0.24  -0.05   0.10
## tipi_3     0.07   -0.05   -0.16   0.09   1.00    0.36    0.12   0.41   0.05
## tipi_8r   -0.16    0.08    0.18   0.01   0.36    1.00    0.31   0.13  -0.17
## tipi_4r    0.09    0.22    0.16  -0.24   0.12    0.31    1.00   0.48  -0.01
## tipi_9     0.14   -0.07   -0.18  -0.05   0.41    0.13    0.48   1.00   0.11
## tipi_5     0.24    0.11    0.03   0.10   0.05   -0.17   -0.01   0.11   1.00
## tipi_10r  -0.04    0.22    0.25   0.03  -0.16    0.23    0.11  -0.26   0.18
```

```
##          tipi_10r
## tipi_1      -0.04
## tipi_6r      0.22
## tipi_2r      0.25
## tipi_7       0.03
## tipi_3      -0.16
## tipi_8r      0.23
## tipi_4r      0.11
## tipi_9      -0.26
## tipi_5       0.18
## tipi_10r     1.00
```

```
# Taiwanese
data %>% dplyr::filter(cultureEA == 0) %>%
  dplyr::select("tipi_1", "tipi_6r",
                "tipi_2r", "tipi_7",
                "tipi_3", "tipi_8r",
                "tipi_4r", "tipi_9",
                "tipi_5", "tipi_10r")%>%
  cor(use = "pairwise.complete.obs") %>%
  round(2)
```

```
##          tipi_1 tipi_6r tipi_2r tipi_7 tipi_3 tipi_8r tipi_4r tipi_9 tipi_5
## tipi_1     1.00    0.68   -0.28   0.00   0.02   -0.31   -0.13  -0.39   0.21
## tipi_6r    0.68    1.00   -0.18   0.07  -0.06   -0.32   -0.10  -0.39   0.11
## tipi_2r   -0.28   -0.18    1.00   0.10   0.08    0.37    0.14   0.32   0.02
## tipi_7     0.00    0.07    0.10   1.00  -0.05    0.06   -0.07   0.04   0.18
## tipi_3     0.02   -0.06    0.08  -0.05   1.00    0.42    0.00   0.27   0.28
## tipi_8r   -0.31   -0.32    0.37   0.06   0.42    1.00    0.10   0.26  -0.05
## tipi_4r   -0.13   -0.10    0.14  -0.07   0.00    0.10    1.00   0.38  -0.03
## tipi_9    -0.39   -0.39    0.32   0.04   0.27    0.26    0.38   1.00   0.06
## tipi_5     0.21    0.11    0.02   0.18   0.28   -0.05   -0.03   0.06   1.00
## tipi_10r   0.17    0.26   -0.02  -0.01  -0.02   -0.18    0.31  -0.03   0.31
##          tipi_10r
## tipi_1       0.17
## tipi_6r      0.26
## tipi_2r     -0.02
## tipi_7      -0.01
## tipi_3      -0.02
## tipi_8r     -0.18
## tipi_4r      0.31
## tipi_9      -0.03
## tipi_5       0.31
## tipi_10r     1.00
```

Interpersonal Reativity Index (IRI)

Next, we compute the composite scores of the participant-level IRI variables. Specifically, we calculate:

1. participant's perspective-taking `iri_pt`

- calculated by averaging `iri3`, `iri8`, `iri11`, `iri15`, `iri21`, `iri25`, & `iri28`

2. participant's fantasy, `iri_fs`

- calculated by averaging `iri1`, `iri5`, `iri7`, `iri12`, `iri16`, `iri23`, & `iri26`

3. participant's empathetic concern, `iri_ec`

- calculated by averaging `iri2`, `iri4`, `iri9`, `iri14`, `iri18`, `iri20`, & `iri22`

4. participant's personal distress, `iri_pd`

- calculated by averaging `iri6`, `iri10`, `iri13`, `iri17`, `iri19`, `iri24`, & `iri27`

```
# compute iri variables
data <- data %>%
  rowwise() %>%
  dplyr::mutate( # reverse code iri items
    iri3r = 6 - iri3,
    iri4r = 6 - iri4,
    iri7r = 6 - iri7,
    iri12r = 6 - iri12,
    iri13r = 6 - iri13,
    iri14r = 6 - iri14,
    iri15r = 6 - iri15,
    iri18r = 6 - iri18,
    iri19r = 6 - iri19,
    ) %>%
  dplyr::mutate( # calculate iri_pt, iri_fs, iri_ec, iri_pd
    iri_pt = mean(
      c(iri3r, iri8, iri11, iri15r, iri21, iri25, iri28), na.rm = T),
    iri_fs = mean(
      c(iri1, iri5, iri7r, iri12r, iri16, iri23, iri26), na.rm = T),
    iri_ec = mean(
      c(iri2, iri4r, iri9, iri14r, iri18r, iri20, iri22), na.rm = T),
    iri_pd = mean(
      c(iri6, iri10, iri13r, iri17, iri19r, iri24, iri27), na.rm = T)
    ) %>%
  ungroup()
```

```
reliab_iri <- c("iri3r, iri8, iri11, iri15r, iri21, iri25, iri28",
               "iri1, iri5, iri7r, iri12r, iri16, iri23, iri26",
               "iri2, iri4r, iri9, iri14r, iri18r, iri20, iri22",
               "iri6, iri10, iri13r, iri17, iri19r, iri24, iri27")

reliab_iri_names <- c("iri_pt", "iri_fs", "iri_ec", "iri_pd")

all_alphas(data, items = reliab_iri, constructs = reliab_iri_names,
           sensitivity = sensitivity_analyses)
```

```
## # A tibble: 12 x 5
##    construct      `!is.na(subID)` `exclude_age != 1` `exclude_sd != 1`
##    <chr>                    <dbl>              <dbl>             <dbl>
##  1 iri_pt_overall            0.77               0.77              0.77
```

```
## 2 iri_fs_overall          0.8              0.8              0.78
## 3 iri_ec_overall          0.79             0.8              0.79
## 4 iri_pd_overall          0.8              0.8              0.79
## 5 iri_pt_EA               0.78             0.79             0.78
## 6 iri_fs_EA               0.83             0.84             0.82
## 7 iri_ec_EA               0.78             0.8              0.78
## 8 iri_pd_EA               0.84             0.84             0.84
## 9 iri_pt_TN               0.76             0.76             0.76
## 10 iri_fs_TN              0.76             0.76             0.76
## 11 iri_ec_TN              0.79             0.8              0.79
## 12 iri_pd_TN              0.75             0.75             0.71
## # i 1 more variable: `exclude_age != 1 & exclude_sd != 1` <dbl>
```

AVI - Ideal and Actual HAP & LAP

Now, we compute the composite scores of the participant-level AVI variables. Specifically, we calculate:

1. participant's actual HAP, `rHAP`

- calculated by averaging `r1_1` (enthusiastic), `r1_7` (excited), `r1_9` (elated), `r2_9` (euphoric)

2. participant's actual LAP, `rLAP`

- calculated by averaging `r1_6` (relaxed), `r3_3` (calm), `r3_9` (peaceful), `r3_10` (serene)

3. participant's ideal HAP, `iHAP`

- calculated by averaging `i1_1` (enthusiastic), `i1_7` (excited), `i1_9` (elated), `i2_9` (euphoric)

4. participant's ideal LAP, `iLAP`

- calculated by averaging `i1_6` (relaxed), `i3_3` (calm), `i3_9` (peaceful), `i3_10` (serene)

5. participant's ipsatized `rHAP`, `rLAP`, `iHAP`, and `iLAP` scores

- calculated by converting ratings using this formula: `ipsatized score = (raw score - individuals' mean ratings across all emotion items) / (individuals' sd across the ratings on all emotion items)`

```r
# compute AVI variables
data <- data %>%
  rowwise() %>%
  dplyr::mutate( # create rHAP, rLAP, iHAP, iLAP
    rHAP = mean(c(r1_1, r1_7, r1_9, r2_9), na.rm = T),
    rLAP = mean(c(r1_6, r3_3, r3_9, r3_10), na.rm = T),
    iHAP = mean(c(i1_1, i1_7, i1_9, i2_9), na.rm = T),
    iLAP = mean(c(i1_6, i3_3, i3_9, i3_10), na.rm = T)) %>%
  dplyr::mutate( # create intermediate variables for ipsatize scores
    rMean = mean(c_across(matches("^r[1-4]_[1-9][0]*$")), na.rm = TRUE),
```

```r
    rSd = sd(c_across(matches("^r[1-4]_[1-9]$|^r4_10$")), na.rm = TRUE),
    iMean = mean(c_across(matches("^i[1-4]_[1-9][0]*$")), na.rm = TRUE),
    iSd = sd(c_across(matches("^i[1-4]_[1-9]$|^i4_10$")), na.rm = TRUE)) %>%
  dplyr::mutate( # create ipsatized item scores
    across(matches("^r[1-4]_[1-9][0]*$"),
           ~ as.numeric((.x - rMean) / rSd),
           .names = "{.col}_i"),
    across(matches("^i[1-4]_[1-9][0]*$"),
           ~ as.numeric((.x - iMean) / iSd),
           .names = "{.col}_i")) %>%
  dplyr::mutate( # create ipsatized rHAP, rLAP, iHAP, iLAP
    rHAP_i = mean(c(r1_1_i, r1_7_i, r1_9_i, r2_9_i), na.rm = T),
    rLAP_i = mean(c(r1_6_i, r3_3_i, r3_9_i, r3_10_i), na.rm = T),
    iHAP_i = mean(c(i1_1_i, i1_7_i, i1_9_i, i2_9_i), na.rm = T),
    iLAP_i = mean(c(i1_6_i, i3_3_i, i3_9_i, i3_10_i), na.rm = T)) %>%
  ungroup()
```

```r
# calculate cronbach alpha

reliab_avi <- c("r1_1, r1_7, r1_9, r2_9",
                "r1_6, r3_3, r3_9, r3_10",
                "i1_1, i1_7, i1_9, i2_9",
                "i1_6, i3_3, i3_9, i3_10")

reliab_avi_names <- c("rHAP", "rLAP", "iHAP", "iLAP")

all_alphas(data, items = reliab_avi, constructs = reliab_avi_names,
           sensitivity = sensitivity_analyses)
```

```
## # A tibble: 12 x 5
##    construct    `!is.na(subID)` `exclude_age != 1` `exclude_sd != 1`
##    <chr>                  <dbl>              <dbl>             <dbl>
##  1 rHAP_overall            0.83               0.81              0.8
##  2 rLAP_overall            0.85               0.85              0.84
##  3 iHAP_overall            0.79               0.79              0.76
##  4 iLAP_overall            0.76               0.77              0.75
##  5 rHAP_EA                 0.87               0.85              0.84
##  6 rLAP_EA                 0.85               0.86              0.83
##  7 iHAP_EA                 0.8                0.79              0.74
##  8 iLAP_EA                 0.73               0.75              0.69
##  9 rHAP_TN                 0.78               0.78              0.77
## 10 rLAP_TN                 0.86               0.86              0.85
## 11 iHAP_TN                 0.77               0.77              0.77
## 12 iLAP_TN                 0.81               0.81              0.81
## # i 1 more variable: `exclude_age != 1 & exclude_sd != 1` <dbl>
```

AVI - Maximizing Positivity

Now, we compute the composite scores of the participant-level AVI variables. Specifically, we calculate:

1. participant's actual PA, `rPA`

- calculated by averaging `r1_1` (enthusiastic), `r1_7` (excited), `r1_9` (elated), `r2_9` (euphoric), `r1_6` (relaxed), `r3_3` (calm), `r3_9` (peaceful), `r3_10` (serene), `r3_1` (happy), `r3_7` (satisfied), `r2_5` (content)

2. participant's actual NA, `rNA`

- calculated by averaging `r2_10` (fearful), `r3_6` (hostile), `r1_3` (nervous), `r2_8` (sad), `r2_2` (lonely), `r3_4` (unhappy), `r2_6` (sluggish), `r1_10` (sleepy), `r1_4` (dull)

3. participant's ideal PA, `iPA`

- calculated by averaging `i1_1` (enthusiastic), `i1_7` (excited), `i1_9` (elated), `i2_9` (euphoric), `i1_6` (relaxed), `i3_3` (calm), `i3_9` (peaceful), `i3_10` (serene), `i3_1` (happy), `i3_7` (satisfied), `i2_5` (content)

4. participant's ideal NA, `iNA`

- calculated by averaging `i2_10` (fearful), `i3_6` (hostile), `i1_3` (nervous), `i2_8` (sad), `i2_2` (lonely), `i3_4` (unhappy), `i2_6` (sluggish), `i1_10` (sleepy), `i1_4` (dull)

5. participant's ipsatized `rPA`, `rNA`, `iPA`, and `iNA` scores
6. participant's raw maximizing positivity scores `maxPos` and ipsatized maximizing positivity scores `maxPos_i`

- calculated by subtracting (ipsatized) ideal NA from ideal PA

```
# compute AVI variables
data <- data %>%
  rowwise() %>%
  dplyr::mutate( # create rPA, rNA, iPA, iNA
    rPA = mean(
      c(r1_1, r1_7, r1_9, r2_9, r1_6, r3_3, r3_9, r3_10, r3_1, r3_7, r2_5),
      na.rm = T),
    rNA = mean(
      c(r2_10, r3_6, r1_3, r2_8, r2_2, r3_4, r2_6, r1_10, r1_4),
      na.rm = T),
    iPA = mean(
      c(i1_1, i1_7, i1_9, i2_9, i1_6, i3_3, i3_9, i3_10, i3_1, i3_7, i2_5),
      na.rm = T),
    iNA = mean(
      c(i2_10, i3_6, i1_3, i2_8, i2_2, i3_4, i2_6, i1_10, i1_4),
      na.rm = T),

    rPA_i = mean(
      c(r1_1_i, r1_7_i, r1_9_i, r2_9_i, r1_6_i, r3_3_i, r3_9_i, r3_10_i,
        r3_1_i, r3_7_i, r2_5_i), na.rm = T),
    rNA_i = mean(
      c(r2_10_i, r3_6_i, r1_3_i, r2_8_i, r2_2_i, r3_4_i, r2_6_i, r1_10_i,
        r1_4_i), na.rm = T),
    iPA_i = mean(
      c(i1_1_i, i1_7_i, i1_9_i, i2_9_i, i1_6_i, i3_3_i, i3_9_i, i3_10_i,
        i3_1_i, i3_7_i, i2_5_i), na.rm = T),
    iNA_i = mean(
```

```r
      c(i2_10_i, i3_6_i, i1_3_i, i2_8_i, i2_2_i, i3_4_i, i2_6_i, i1_10_i,
        i1_4_i), na.rm = T)) %>%
  dplyr::mutate(
    # create maxPos variables
    maxPos = iPA - iNA,
    maxPos_i = iPA_i - iNA_i) %>%
  ungroup()
```

```r
# calculate cronbach alpha

reliab_avimp <-
  c("r1_1, r1_7, r1_9, r2_9, r1_6, r3_3, r3_9, r3_10, r3_1, r3_7, r2_5",
    "r2_10, r3_6, r1_3, r2_8, r2_2, r3_4, r2_6, r1_10, r1_4",
    "i1_1, i1_7, i1_9, i2_9, i1_6, i3_3, i3_9, i3_10, i3_1, i3_7, i2_5",
    "i2_10, i3_6, i1_3, i2_8, i2_2, i3_4, i2_6, i1_10, i1_4")

reliab_avi_namesmp <- c("rPA", "rNA", "iPA", "iNA")

all_alphas(data, items = reliab_avimp, constructs = reliab_avi_namesmp,
           sensitivity = sensitivity_analyses)
```

```
## # A tibble: 12 x 5
##    construct   `!is.na(subID)` `exclude_age != 1` `exclude_sd != 1`
##    <chr>                 <dbl>              <dbl>             <dbl>
##  1 rPA_overall            0.87               0.87              0.86
##  2 rNA_overall            0.86               0.86              0.85
##  3 iPA_overall            0.84               0.84              0.84
##  4 iNA_overall            0.88               0.88              0.83
##  5 rPA_EA                 0.9                0.9               0.89
##  6 rNA_EA                 0.89               0.9               0.89
##  7 iPA_EA                 0.84               0.85              0.83
##  8 iNA_EA                 0.93               0.94              0.89
##  9 rPA_TN                 0.85               0.85              0.84
## 10 rNA_TN                 0.83               0.83              0.83
## 11 iPA_TN                 0.85               0.85              0.86
## 12 iNA_TN                 0.77               0.76              0.77
## # i 1 more variable: `exclude_age != 1 & exclude_sd != 1` <dbl>
```

Ipsatized giving scores

Finally, we compute the ipsatized scores of the trials. Alike the AVI variables, we calculate this by converting the amount of money given using this formula: `ipsatized score = (raw amoung given - individuals' mean amount given across all trials) / (individuals' sd across the amount of money given across all trials)`

```r
# compute ipsatized giving scores
data <- data %>%
  rowwise() %>%
  dplyr::mutate( # create intermediate variables for calculating ipsatize scores
    givingMean = mean(
      c_across(matches("^raw[1-9][1-9]*[0]*$")), na.rm = TRUE),
```

```r
    givingSd = sd(
      c_across(matches("^raw[1-9][1-9]*[0]*$")), na.rm = TRUE)) %>%
  dplyr::mutate( # create ipsatized item scores
    across(matches("^raw[1-9][1-9]*[0]*$"),
           ~ as.numeric((.x - givingMean) / givingSd),
           .names = "{.col}_i")) %>%
  ungroup()
```

Ipsatized trait rating scores

Finally, we compute the ipsatized scores of the trait ratings for the faces (items that start with `trait`, `vale`, and `arou`).

All participants rated the faces on numerous traits. Specifically, they responded to the prompt "To what extent is this person…" (e.g., "Competent", "Trustworthy", etc.; 1 = *Not at all*, 4 = *Moderately*, 7 = *Extremely*) and how positive ("*How negative - positive do you feel when you see this face?*"; 1 = *Extremely negative*, 7 = *Extremely positive*) and aroused ("*How not-aroused - aroused do you feel when you see this face? Aroused means activated, charged, alert, or energized physically and mentally.*"; 1 = *Not at all aroused*, 7 = *Extremely aroused*) they feel when they see the face. For the repeated ratings, we only include the first rating they provide.

Alike the AVI and giving variables, we calculate this by converting the ratings given using this formula: `ipsatized score = (raw rating - individuals' mean rating across all traits and faces) / (individuals' sd rating across all traits and faces)`. Note that for valence (`vale`) and arousal (`arou`) ratings, they are ipsatized within the valence and arousal ratings only (i.e., `ipsatized valence = (raw valence rating - individuals' mean rating of valence across all faces) / (individuals' sd of valence across all faces)`.

```r
# compute ipsatized trait rating scores
data <- data %>%
  rowwise() %>%
  dplyr::mutate( # create intermediate variables for calculating ipsatize scores
    traitRatingMean = mean(
      c_across(matches("^trait[1-9][0-9]*_")), na.rm = TRUE),
    traitRatingSd = sd(
      c_across(matches("^trait[1-9][0-9]*_")), na.rm = TRUE),
    valeRatingMean = mean(
      c_across(matches("^vale[1-9][0-9]*$")), na.rm = TRUE),
    valeRatingSd = sd(
      c_across(matches("^vale[1-9][0-9]*$")), na.rm = TRUE),
    arouRatingMean = mean(
      c_across(matches("^arou[1-9][0-9]*$")), na.rm = TRUE),
    arouRatingSd = sd(
      c_across(matches("^arou[1-9][0-9]*$")), na.rm = TRUE)) %>%
  dplyr::mutate(
    # create ipsatized item scores
    across(matches("^trait[1-9][0-9]*_"),
           ~ as.numeric((.x - traitRatingMean) / traitRatingSd),
           .names = "{.col}_i")) %>%
  dplyr::mutate( # create ipsatized item scores
    across(matches("^vale[1-9][0-9]*$"),
           ~ as.numeric((.x - valeRatingMean) / valeRatingSd),
           .names = "{.col}_i")) %>%
```

```r
  dplyr::mutate( # create ipsatized item scores
    across(matches("^arou[1-9][0-9]*$"),
           ~ as.numeric((.x - arouRatingMean) / arouRatingSd),
           .names = "{.col}_i")) %>%
  ungroup()
```

Save data frame

```r
write.csv(data, "data_computed.csv")
```